

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №4

Тема: Методи розв'язування нелінійних рівнянь. Метод поділу відрізка навпіл, метод хорд

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати постановку задачі відокремлення коренів нелінійного рівняння, навчитися застосовувати метод поділу відрізка навпіл (бісекції) та метод хорд для наближеного розв'язування нелінійних рівнянь вручну та програмно, набути навичок оцінки точності й кількості ітерацій.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal), калькулятор.

Короткі теоретичні відомості

1. Постановка задачі. Потрібно знайти наближений корінь рівняння $f(x) = 0$ з заданою точністю ϵ , якщо відомо, що на відрізку $[a, b]$ існує рівно один корінь. Умова існування кореня (та зміни знаку) на відрізку — це неперервність $f(x)$ на $[a, b]$ і виконання нерівності $f(a) \cdot f(b) < 0$ (теорема Больцано–Коші).

2. Відокремлення коренів — попередній етап, на якому відрізок $[a, b]$ ділять на дрібніші підвідрізки й знаходять ті з них, на кінцях яких функція набуває значень протилежних знаків. Це можна зробити табулюванням функції з деяким кроком h та аналізом зміни знаку $f(x)$.

3. Метод поділу відрізка навпіл (бісекції).

Алгоритм:

1. Перевірити $f(a) \cdot f(b) < 0$.
2. Обчислити середину $c = (a + b) / 2$.
3. Якщо $f(a) \cdot f(c) < 0$ — корінь у $[a, c]$, покласти $b = c$; інакше — корінь у $[c, b]$, покласти $a = c$.
4. Повторювати кроки 2–3, доки $(b - a) / 2 > \epsilon$ (або $|f(c)| < \epsilon$).
5. Наближений корінь — остання обчислена середина c .

Похибка після n ітерацій: $|x_n - x^*| \leq (b - a) / 2^{n+1}$. Звідси кількість ітерацій, необхідних для досягнення точності ϵ :

$$n \geq \log_2((b - a) / \epsilon)$$

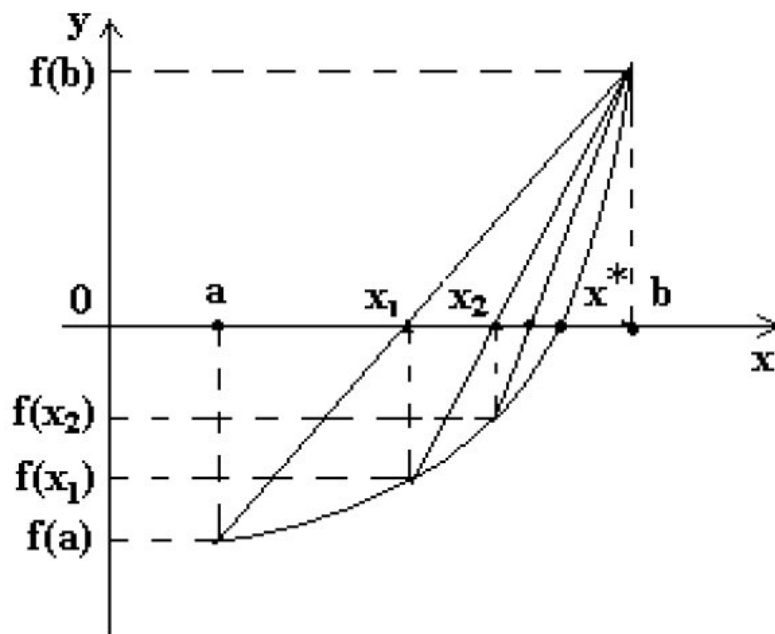
Метод завжди збігається (лінійна швидкість збіжності), але порівняно повільно.

4. Метод хорд (метод фальшивого положення, regula falsi).

Замість середини відрізка обчислюється точка перетину хорди (прямої, що з'єднує $(a, f(a))$ і $(b, f(b))$) з віссю x :

$$x = a - f(a) \cdot \frac{b - a}{f(b) - f(a)}$$

$$x = a - f(a) \cdot \frac{b - a}{f(b) - f(a)}$$



Алгоритм:

1. Перевірити $f(a) \cdot f(b) < 0$.
2. Обчислити x за формулою вище.
3. Якщо $f(a) \cdot f(x) < 0$ — покласти $b = x$; інакше — покласти $a = x$.
4. Повторювати кроки 2-3, доки $|f(x)| < \epsilon$ (або $|x_n - x_{n-1}| < \epsilon$).

На відміну від бісекції, у методі хорд один із кінців відрізка може «застрягти» (якщо функція опукла/угнута в один бік), тому швидкість збіжності зазвичай вища за лінійну, але не квадратична.

5. Критерії зупинки ітераційного процесу: $|x_{n+1} - x_n| < \epsilon$, або $|f(x_n)| < \epsilon$, або досягнення заданої максимальної кількості ітерацій.

Хід роботи

Завдання 1. Ручний розрахунок методом бісекції

Дано рівняння $x^3 - x - 1 = 0$ на відрізку $[1; 2]$. Знайти наближений корінь із точністю $\epsilon = 0,01$ методом поділу відрізка навпіл.

Порядок виконання:

1. Обчисліть $f(1) = 1^3 - 1 - 1 = -1$ та $f(2) = 2^3 - 2 - 1 = 5$.
Переконайтесь, що $f(1) \cdot f(2) < 0$ – корінь на відрізку існує.
2. Побудуйте таблицю зі стовпцями: № ітерації, a , b , $c = (a+b)/2$, $f(a)$, $f(b)$, $f(c)$, $b-a$.
3. На кожній ітерації обчисліть $c = (a+b)/2$ та $f(c)$ (підставте c у рівняння $x^3 - x - 1$).
4. Порівняйте знак $f(a) \cdot f(c)$: якщо добуток від'ємний — замініть b на c ; якщо додатний — замініть a на c .
5. Занесіть новий рядок у таблицю з оновленими межами.
6. Повторюйте кроки 3–5, доки довжина відрізка $(b-a)$ не стане меншою за $2\epsilon = 0,02$.
7. Порахуйте, скільки ітерацій знадобилось фактично, і порівняйте з теоретичною оцінкою $n \geq \log_2((2-1)/0,01) \approx 6,64$, тобто 7 ітерацій.
8. Запишіть остаточну відповідь як середину останнього відрізка, округливши до точності ϵ .

Завдання 2. Ручний розрахунок методом хорд

Для того самого рівняння $x^3 - x - 1 = 0$ на відрізку $[1; 2]$ знайти корінь із точністю $\epsilon = 0,001$ методом хорд.

Порядок виконання:

1. Побудуйте таблицю зі стовпцями: № ітерації, a , b , x (за формулою хорди), $f(x)$.
2. Обчисліть перше наближення $x_1 = a - f(a) \cdot (b-a) / (f(b)-f(a))$,
підставивши $a=1$, $b=2$, $f(a)=-1$, $f(b)=5$.
3. Обчисліть $f(x_1)$, підставивши x_1 у рівняння.
4. Визначте, який кінець відрізка замінити: якщо $f(a) \cdot f(x_1) < 0$ — покладіть $b = x_1$ (лівий кінець залишається нерухомим); інакше — покладіть $a = x_1$.
5. Повторіть кроки 2–4 для наступних ітерацій, щоразу перераховуючи x за формулою хорди з оновленими a , b .
6. Зупиніться, коли $|f(x)| < 0,001$ або різниця сусідніх наближень $|x_n - x_{n-1}| < 0,001$.
7. Порівняйте кількість ітерацій, витрачених методом хорд, із кількістю ітерацій методу бісекції з завдання 1 (за порівнянню точність). Зробіть висновок, який метод збігається швидше для цього рівняння.

Завдання 3. Програмна реалізація методу бісекції

Реалізувати мовою Python функцію `bisection(f, a, b, eps)`, яка повертає наближений корінь і кількість ітерацій. Перевірити на рівнянні із завдання 1.

Порядок виконання:

1. Оголосіть функцію $f(x)$, що обчислює ліву частину рівняння $x^3 - x - 1$.

2. Напишіть функцію `bisection(f, a, b, eps)` зі змінною-лічильником ітерацій `n = 0`.
3. У циклі `while (b - a) / 2 > eps`: обчисліть `c = (a + b) / 2`.
4. Перевірте умову `f(a) * f(c) < 0`: якщо істинна — оновіть `b = c`, інакше — `a = c`.
5. Збільште лічильник ітерацій `n += 1`.
6. Після виходу з циклу поверніть `c = (a + b) / 2` та кількість ітерацій `n`.
7. Викличте функцію з параметрами `a=1, b=2, eps=1e-6` та виведіть результат.
8. Порівняйте отримане програмою число ітерацій із теоретичною формулою $n \geq \log_2((b-a)/\epsilon)$.

Приклад реалізації мовою Python:

```
python
def f(x):
    return x**3 - x - 1

def bisection(f, a, b, eps):
    n = 0
    while (b - a) / 2 > eps:
        c = (a + b) / 2
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
        n += 1
    return (a + b) / 2, n

root, iterations = bisection(f, 1, 2, 1e-6)
print(f"Корінь = {root:.7f}, кількість ітерацій = {iterations}")
```

Результат виконання програми:

Корінь = 1.3247175, кількість ітерацій = 19

Отримане значення кореня ($\approx 1,3247175$) практично збігається з точним коренем рівняння ($1,3247180$), що підтверджує коректність реалізації.

Завдання 4. Програмна реалізація методу хорд та порівняння методів

Реалізувати функцію `chord(f, a, b, eps)` методом хорд, порівняти кількість ітерацій з методом бісекції для однакової точності `eps = 1e-6` та побудувати графік збіжності обох методів (похибка від номера ітерації, логарифмічна шкала).

Порядок виконання:

1. Напишіть функцію `chord(f, a, b, eps)` за аналогією з `bisection`, але з обчисленням $x = a - f(a) * (b - a) / (f(b) - f(a))$ замість середини відрізка.
2. У циклі перевіряйте умову зупинки `abs(f(x)) < eps`, а не довжину відрізка.
3. Визначте, яку межу оновлювати (`a` чи `b`), за знаком добутку `f(a) * f(x)` — аналогічно бісекції.
4. Додайте лічильник ітерацій та захист від зациклення (обмеження максимальної кількості ітерацій, наприклад, 200).
5. Викличте обидві функції (`bisection` і `chord`) з однаковими параметрами (`a=1, b=2, eps=1e-6`) і виведіть кількість ітерацій кожної — порівняйте числа.

- Для побудови графіка збіжності модифікуйте обидві функції так, щоб вони повертали історію наближень на кожній ітерації (список значень x або s).
- Обчисліть похибку на кожній ітерації як модуль різниці між наближенням і точним коренем (для еталонного значення можна взяти корінь, обчислений із дуже малим ϵ , наприклад $1e-12$).
- Побудуйте графік `plt.semilogy(...)` (похибка на логарифмічній осі Y , номер ітерації на осі X) для обох методів на одному малюнку.
- Проаналізуйте графік: яка лінія спадає швидше і чому.

Приклад реалізації мовою Python:

```
python
def chord(f, a, b, eps):
    n = 0
    while True:
        x = a - f(a) * (b - a) / (f(b) - f(a))
        if abs(f(x)) < eps:
            return x, n
        if f(a) * f(x) < 0:
            b = x
        else:
            a = x
        n += 1

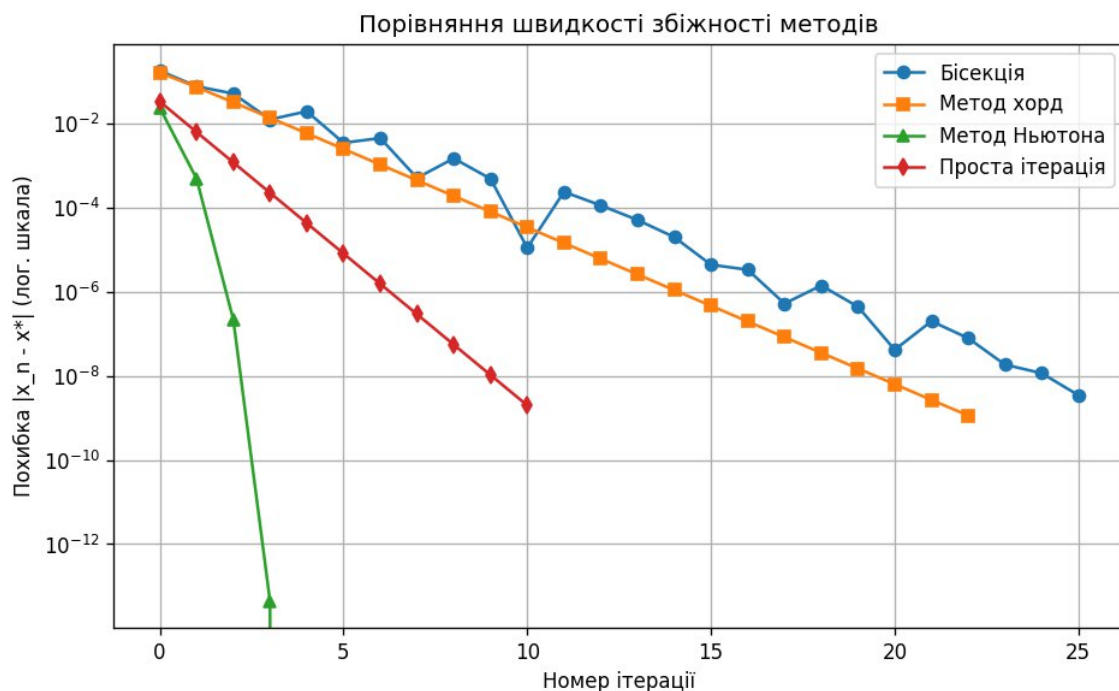
root_bis, n_bis = bisection(f, 1, 2, 1e-6)
root_chord, n_chord = chord(f, 1, 2, 1e-6)

print(f"Бісекція: корінь = {root_bis:.7f}, ітерацій = {n_bis}")
print(f"Метод хорд: корінь = {root_chord:.7f}, ітерацій = {n_chord}")
```

Результат виконання програми:

Бісекція: корінь = 1.3247175, ітерацій = 19
 Метод хорд: корінь = 1.3247178, ітерацій = 16

Графік порівняння швидкості збіжності (разом із методами Ньютона й простої ітерації) наведено на рисунку нижче:



Висновок за результатами: для рівняння $x^3 - x - 1 = 0$ метод хорд досягає заданої точності за 16 ітерацій проти 19 у методу бісекції – це наочно демонструє, що метод хорд, використовуючи інформацію про нахил хорди (а не лише знак функції), збігається швидше, хоча обидва методи мають гарантовану збіжність за умови $f(a) \cdot f(b) < 0$.

Контрольні питання

1. У чому полягає задача відокремлення коренів рівняння?
2. Сформулюйте умову існування кореня на відрізку $[a, b]$ (теорема Больцано–Коші).
3. Опишіть алгоритм методу поділу відрізка навпіл.
4. Виведіть формулу оцінки похибки методу бісекції після n ітерацій.
5. Чим відрізняється формула методу хорд від формули середини відрізка в бісекції?
6. Які критерії зупинки ітераційного процесу застосовують у обох методах?
7. Який із двох методів (бісекції чи хорд) зазвичай збігається швидше і чому?
8. Які недоліки має метод бісекції порівняно з методом хорд?

Зміст звіту

1. Тема, мета роботи.
2. Розгорнуті ручні розрахунки (таблиці) до завдань 1 і 2.
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Графік порівняння збіжності методів.
5. Висновки.